Test 4 Review

Dynamics, Decision Making, the LMU, Learning, and Analysing Representation

Lectures 6–9 and Practice Notebook 4 · Thursday, November 12

How to use this review

Each section restates the course content in the lectures' own notation, then works through the derivations you are expected to reproduce by hand. Boxes mark the parts that matter most:

- **Key result** boxes hold what you should be able to write from memory.
- **Derivation** boxes are step-by-step. Practise reproducing them on blank paper.
- **Worked example** boxes are test-style questions with answers.
- **Trap** boxes flag sign conventions, notation clashes and typos in the source notes.

Section 8 is a one-page formula sheet and sketch checklist for the morning of the test.

Section	Source	Main skills
1. Representation recap	L03–L05	Encoders, decoders, radius, intercepts, γ and Υ
2. Recurrent dynamics	L06	Synapse ODE, $A' = \tau A + I$, reading off A and B, fixed points
3. Decision making	PN4 §1	Accumulator, saturation, drift-diffusion, choice decoding, intercepts
4. Functions and the LMU	L07, PN4 §2	Nyquist, bases, Legendre, Padé, delay decoding, classification
5. Supervised learning	L08, PN4 §3–4	Delta rule, PES, error sign, learning curves, adaptive control
6. Unsupervised learning	L08	PCA, Hebb, Oja, BCM, STDP
7. Analysing representation	L09, PN4 §5	SVD of γ, χ and Φ, Legendre-like basis, noise, $S_m > \sigma^2$

1 Representation recap (what the rest builds on)

A population of n neurons represents a value $\mathbf{x}$ (scalar or vector). Each neuron i has an encoder $\mathbf{e}_i$ (a unit vector; ± 1 in 1-D), a gain α_i and a bias current J_i^{bias}:

$$a_i(\mathbf{x}) = G_i\Big[\alpha_i \langle \mathbf{e}_i, \mathbf{x}\rangle + J_i^{\text{bias}}\Big].$$

The **intercept** is the value of $\langle \mathbf{e}_i, \mathbf{x}\rangle$ at which neuron i starts to fire. The **radius** r is the range the population is built for: evaluation points are drawn from $\|\mathbf{x}\| \le r$, and nothing constrains the decoders outside it.

Decoding is linear: $\hat{\mathbf{x}} = \sum_i \mathbf{d}_i\, a_i(\mathbf{x})$. Stack M evaluation points as rows to get the activity matrix $\mathbf{A}$ ($M \times n$, samples $\times$ neurons, as in L09) and the targets $\mathbf{X}$. The least-squares decoders are

$$\mathbf{D} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{X} = \gamma^{-1}\Upsilon, \qquad \gamma = \mathbf{A}^T\mathbf{A}\ (n \times n), \quad \Upsilon = \mathbf{A}^T\mathbf{X}.$$

With noise, add a regularizer to the diagonal, $\gamma \to \gamma + \lambda\mathbf{I}$, with λ proportional to the noise variance.

Decoding a function $f(\mathbf{x})$ uses the same $\mathbf{A}$ and the targets $f(\mathbf{x})$, so only Υ changes; γ is shared by every function. This fact drives Section 7. **Weights** between populations are $w_{ij} = \alpha_i \langle \mathbf{e}_i, \mathbf{d}_j \rangle$.

Trap: the letter A

Lecture 6 uses $\mathbf{A}$ for the dynamics matrix in $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$. Lecture 9 uses $\mathbf{A}$ for the activity matrix. PN4 also writes the activity matrix as neurons $\times$ samples, so its Gram matrix is $\mathbf{A}\mathbf{A}^T$. Either way the Gram matrix is $n \times n$ (neurons $\times$ neurons). State your convention in one line.

2 Recurrent dynamics (Lecture 6)

2.1 What a synapse does

A spike arriving at a first-order synapse produces a current that jumps and then decays:

$$h(t) = \begin{cases} \frac{1}{\tau} e^{-t/\tau}, & t \geq 0, \\ 0, & t < 0, \end{cases} \qquad \int_0^\infty h(t)\, dt = 1.$$

The $1/\tau$ normalizes the area to 1, so the filter does not change a constant signal's size. If the incoming (decoded) signal is $v(t)$, the receiving population is driven by the filtered signal $y = h * v$, a weighted average of past inputs in which older inputs fade by a factor of e every τ seconds.

Derivation: the synapse ODE $\tau\dot{y} = -y + v$

Route 1 (time domain). Write the convolution with t in the upper limit:

$$y(t) = \int_{-\infty}^{t} \frac{1}{\tau} e^{-(t-t')/\tau} v(t')\, dt'.$$

Differentiate with respect to t. The upper limit contributes the integrand at $t' = t$, which is $\frac{1}{\tau} v(t)$. Differentiating the exponential brings down $-\frac{1}{\tau}$ and leaves $y(t)$. So

$$\dot{y} = \frac{v}{\tau} - \frac{y}{\tau} \quad \Longrightarrow \quad \boxed{\tau\dot{y} = -y + v}.$$

Route 2 (Laplace, as in L06). $H(s) = \mathcal{L}\{h\} = \int_0^\infty \frac{1}{\tau} e^{-t/\tau} e^{-st} dt = \dfrac{1}{1 + s\tau}$. Then

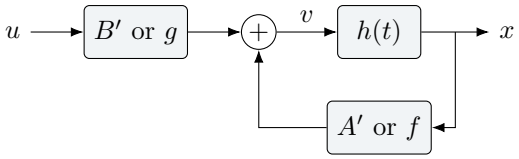
$$Y(s) = \frac{V(s)}{1 + s\tau} \iff Y + s\tau Y = V \iff y + \tau\dot{y} = v,$$

because multiplication by s is a time derivative.

Why this is the population's value. The neurons are driven by y, and the NEF assumes they represent what that current encodes, so the decoded value $x \approx y$. The synapse is the slow part that sets the dynamics.

Sanity check. For constant v and $x(0) = 0$, $x(t) = v(1 - e^{-t/\tau})$. With no recurrence, a population is a low-pass filter: x chases its input with time constant τ. The $-x$ term is the "forgetting."

2.2 Adding a recurrent connection



The input connection computes $g(u)$ (transform B' in the linear case) and the recurrent connection computes $f(x)$ (transform A'). Both pass through the same synapse h, so they add before filtering: $v = g(u) + f(x)$, and $x = h * (g(u) + f(x))$. Substituting into the synapse ODE gives the boxed equation in L06:

Key result: recurrent connection differential equation (L06)

$$\frac{dx}{dt} = \frac{1}{\tau}\Big(g(u(t)) + f(x(t)) - x(t)\Big).$$

2.3 Linear systems: where A and B come from

Two pairs of letters. A and B are the coefficients of the *dynamics* $\dot{x} = Ax + Bu$, which is the behaviour you want or get. A' and B' are the transforms on the two *connections*, which is what you build. Design goes from (A, B) to (A', B'); analysis goes the other way. The algebra is the same.

Derivation: $A' = \tau A + I$ and $B' = \tau B$

1. Start from the synapse ODE: $\tau\dot{x} = -x + v$.
2. What arrives through the synapse: $v = A'x + B'u$.
3. Substitute and group the x terms: $\tau\dot{x} = -x + A'x + B'u = (A' - 1)x + B'u$.
4. Divide by τ so that $\dot{x}$ stands alone: $\dot{x} = \dfrac{A' - 1}{\tau}x + \dfrac{B'}{\tau}u$.
5. Match against $\dot{x} = Ax + Bu$ term by term:

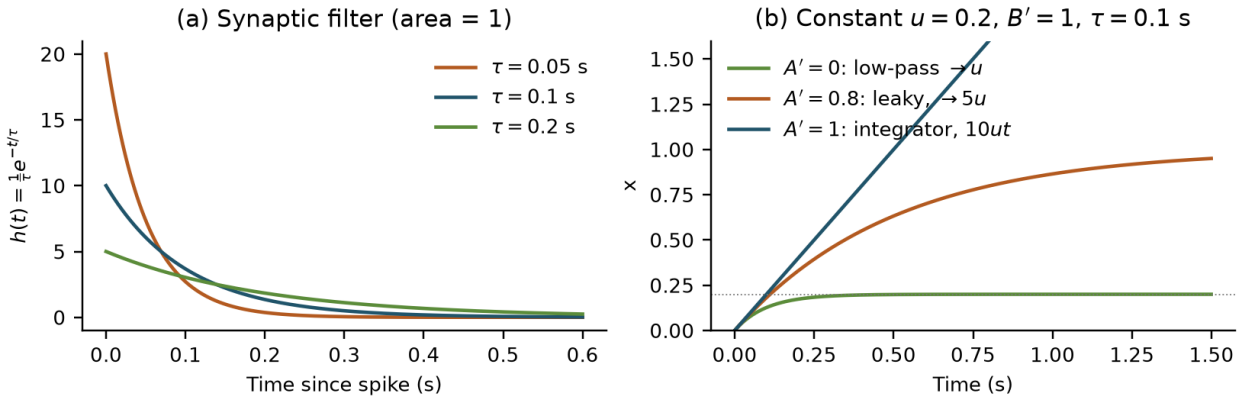
$$A = \frac{A' - I}{\tau}, \qquad B = \frac{B'}{\tau} \qquad \Longleftrightarrow \qquad A' = \tau A + I, \qquad B' = \tau B.$$

L06 does the same steps in the Laplace domain: $X = H(A'X + B'U) \Rightarrow sX = \frac{1}{\tau}(A' - I)X + \frac{1}{\tau}B'U$.

How to read off A and B. A is the coefficient of x and B the coefficient of u, but only after (i) including the synapse's own $-x$, and (ii) dividing by τ so the left side is $\dot{x}$ alone. The recurrent transform is not A: $A' = 1$ gives $A = 0$, because the leak cancels it.

A'	B'	grouped: $\tau\dot{x} =$	A	B	behaviour ($\tau = 0.1$ s)
1	$\tau = 0.1$	$0 \cdot x + 0.1u$	0	1	integrator $\dot{x} = u$ (the L06 design)
1	1	$0 \cdot x + u$	0	10	integrator, 10× faster (PN4 §1)
0.8	1	$-0.2x + u$	-2	10	leaky integrator, settles at $x = 5u$
0	1	$-x + u$	-10	10	plain low-pass, time constant τ
$1 - \tau/\tau_d$	τ/τ_d		$-1/\tau_d$	$1/\tau_d$	low-pass with any τ_d (even faster than τ)
$\begin{pmatrix} 1 & -\tau\omega \\ \tau\omega & 1 \end{pmatrix}$	0		$\begin{pmatrix} 0 & -\omega \\ \omega & 0 \end{pmatrix}$	0	oscillator at ω rad/s

Note the fourth row: even with no recurrence, $A = -1/\tau$. The synapse alone is a leaky integrator.



(a) The synapse's impulse response: a smaller τ forgets faster. (b) The same input with three recurrent transforms, the first rows of the table: no recurrence gives a low-pass filter that settles at u; $A' = 0.8$ leaves part of the leak and settles at $5u$; $A' = 1$ cancels the leak and ramps forever.

Worked example: Practice Test Q1(a)

An identity recurrence through $\tau = 0.1$ s, input with transform 1, matched synapses, $x(0) = 0$. Find A, B and $x(t)$ for constant u.

Synapse: $\tau\dot{x} = -x + A'x + B'u \Rightarrow A = (A' - 1)/\tau$, $B = B'/\tau$. With $A' = 1$, $B' = 1$, $\tau = 0.1$: $A = 0$, $B = 10$ s^{-1}, so $\dot{x} = 10u$ and $x(t) = 10ut$. Equivalently, the boxed L06 equation with $f(x) = x$, $g(u) = u$ gives $\dot{x} = \frac{1}{\tau}(u + x - x) = u/\tau$. The ideal x reaches ± 1 at $t = 1/(10|u|)$: 0.5 s for $|u| = 0.2$ and 1 s for $|u| = 0.1$.

2.4 Nonlinear systems and the notation switch

If the recurrence computes a nonlinear f, there are no matrices; keep the boxed form and analyse it directly. For example $f(x) = x^2$ gives $\dot{x} = \frac{1}{\tau}(g(u) + x^2 - x)$.

Design formula (L06, "additive" systems). To implement a target $\dot{x} = f(x) + g(u)$, put $f'(x) = \tau f(x) + x$ on the recurrent connection and $g'(u) = \tau g(u)$ on the input connection. It is the same derivation as the linear case, with whole functions in place of $A'x$ and $B'u$. The network obeys $\dot{x} = \frac{1}{\tau}(g'(u) + f'(x) - x)$; matching the x terms gives $\frac{1}{\tau}(f'(x) - x) = f(x)$, and matching the u terms gives $\frac{1}{\tau}g'(u) = g(u)$. Multiply by τ to undo the synapse's $1/\tau$, and add x to cancel its leak. With $f(x) = Ax$ and $g(u) = Bu$ this is exactly $A' = \tau A + I$, $B' = \tau B$. The prime is a *label* ("the version on the connection"), not a derivative.

Worked example: designing a bistable decision

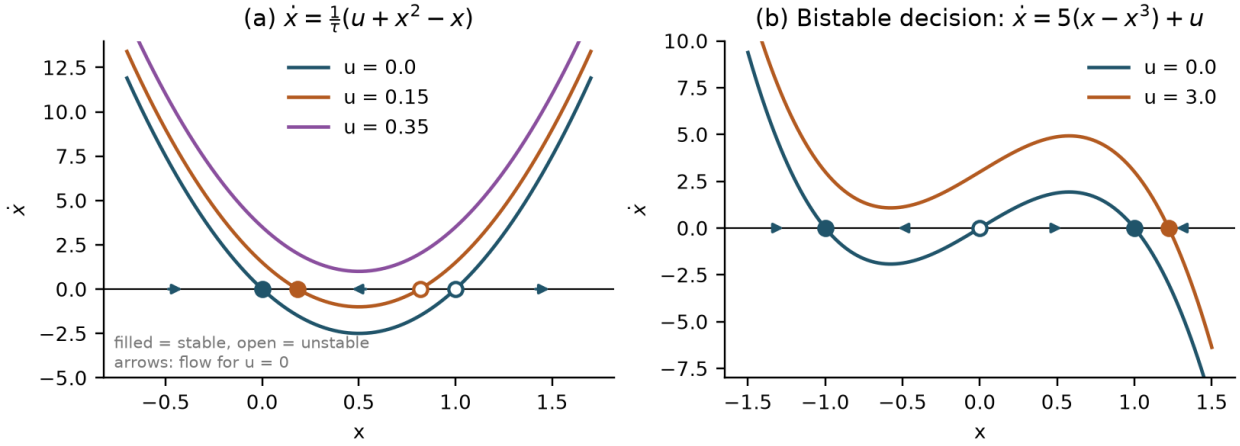
Target: $\dot{x} = 5(x - x^3) + u$, with stable states at ± 1 and an unstable state at 0 (see §2.5). With $\tau = 0.1$: recurrent connection $f'(x) = 0.1 \cdot 5(x - x^3) + x = 1.5x - 0.5x^3$; input connection $g'(u) = 0.1u$. *Check:* $\dot{x} = \frac{1}{0.1}(0.1u + 1.5x - 0.5x^3 - x) = u + 5x - 5x^3$. *Analysis direction:* given the connections, $f = (f' - x)/\tau$ and $g = g'/\tau$. For PN4 §1, $f'(x) = x$ and $g'(u) = u$ give $f = 0$, $g = u/\tau$, so $\dot{x} = 10u$, as in Q1(a).

Trap: L06 flips what f and g mean

In the boxed recurrent equation, unprimed f and g are the functions on the *connections*. In the "additive" section, unprimed f and g are the *target* dynamics and the primed ones are the connections. The rule that always holds: *connection* $= \tau\times$ *target* + *identity feedback*. The notes print $g'(u) = \tau u$; it should read $\tau g(u)$ (they agree only if $g(u) = u$). On line 333, $G(X(s))$ should be $G(U(s))$.

2.5 Fixed points and phase portraits

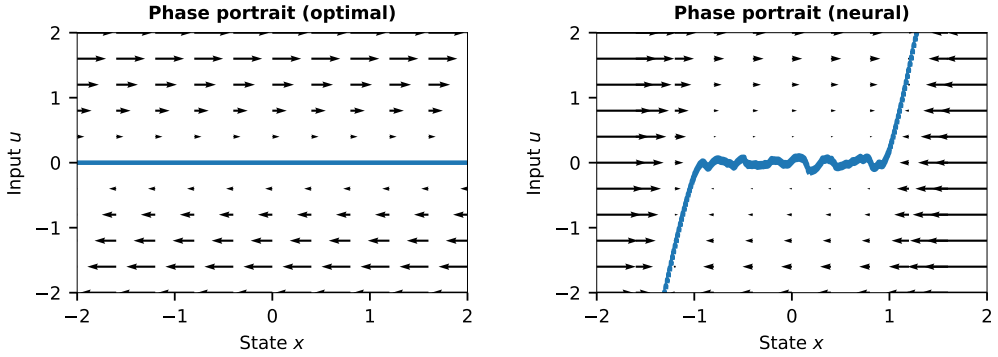
A **phase portrait** plots $\dot{x}$ against x. Fixed points are where the curve crosses $\dot{x} = 0$. Where the curve is above the axis, x increases (flow to the right); below, it decreases (flow to the left). A fixed point is **stable** if $\dot{x}$ crosses zero from positive to negative, so the arrows on both sides point into it.



Reading panel (a), the $f(x) = x^2$ example from §2.4 with $g(u) = u$. For $u = 0$, $\dot{x} = x(x-1)/\tau$ is zero at $x = 0$ and $x = 1$. Just below 0 it is positive and just above 0 it is negative, so $x = 0$ is stable; around $x = 1$ the flow points away, so $x = 1$ is unstable. Raising u lifts the parabola: the two fixed points $x^* = \frac{1 \pm \sqrt{1-4u}}{2}$ move together, merge at $u = \frac{1}{4}$, and for $u > \frac{1}{4}$ (purple) there are none, so x grows until the population saturates.

Reading panel (b), the bistable decision designed in §2.4. With $u = 0$ there are stable points at ± 1 and an unstable point at 0: a memory that holds one of two values. A strong input ($u = 3$) lifts the curve so only the positive stable point survives; the input has made the decision. Compare the PN4 accumulator, whose "stopping" near ± 1 is an accident of the radius rather than designed fixed points.

Neural integrators drift. A perfect integrator with $u = 0$ has $\dot{x} \equiv 0$, a line of fixed points: every value is remembered. A neural integrator has small static decoding errors, $\hat{f}(x) - x \neq 0$, so its phase portrait wiggles around zero. The line breaks into a few stable points, and the stored value *drifts* toward them (L06):



L06 figure. Each point is a state x (horizontal) under a constant input u (vertical); arrows show the flow, and the blue curve is where $\dot{x} = 0$. Ideal integrator: $\dot{x} = 0$ only when $u = 0$, for every x. Neural integrator: the blue curve wiggles, so with $u = 0$ only a few values of x are fixed points, and between them the state slowly drifts.

Neural implementations of dynamics are always approximations.

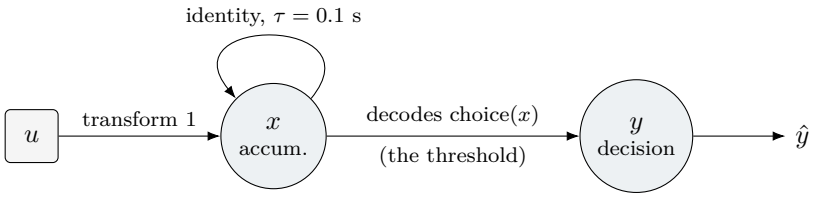
3 Accumulate-to-threshold decision making (PN4 §1)

The model. A 1-D, 50-neuron ensemble with radius 1, a recurrent identity connection through $\tau = 0.1$ s, and a constant evidence input u with transform 1. Section 2 gives $\dot{x} = 10u$ and $x(t) = 10ut$: the network *integrates* evidence, so its value is proportional to evidence × time. Stronger evidence ramps faster.

Trap: the input is not scaled by τ

The textbook integrator uses $B' = \tau$ to get $\dot{x} = u$. PN4 uses transform 1, so the accumulator runs 10× faster. PN4 also leaves the input synapse at Nengo's default (5 ms), so the mapping is approximate; the mismatch mainly affects the start of the ramp. Practice Test Q1 asks you to treat the synapses as matched.

3.1 The whole model in equations



Two populations in series. x *integrates* the evidence; the connection from x to y *applies the threshold* (its decoders compute choice(x)); y *represents the decision* (0 = undecided, ±1 = chose ±). y has no recurrence, so it reports choice(x) from moment to moment; the decision persists only because x stays beyond ±0.9.

Key result: the decision model, level by level

1. Ideal (no neurons). Evidence is integrated, then thresholded:

$$\dot{x} = \frac{u}{\tau} = 10u, \qquad x(t) = 10ut, \qquad y(t) = \text{choice}(x(t)) = \begin{cases} +1, & x > 0.9, \\ -1, & x < -0.9, \\ 0, & \text{otherwise.} \end{cases}$$

Decision time solves $x(t^*) = 0.9$: $t^* = \dfrac{0.9\,\tau}{|u|} = \dfrac{0.09}{|u|}$. **Choice** = sign($u$). So $y(t) = \text{sign}(u)$ for $t > t^*$ and 0 before.

2. With neurons. Population x has 50 neurons $a_i(x) = G_i[\alpha_i e_i x + J_i^{\text{bias}}]$.

- Recurrent connection: decoders $\mathbf{d}^{\text{id}}$ fit to the identity on $[-1, 1]$, giving $\hat{f}(x) = \sum_i d_i^{\text{id}} a_i(x)$. Through the synapse,

  $$\tau\dot{x} = -x + \hat{f}(x) + u.$$

  Inside the radius $\hat{f}(x) \approx x$, so $\dot{x} \approx u/\tau$; beyond it $\hat{f}(x) < x$, so x settles where $x - \hat{f}(x) = u$ (§3.2).
- Connection $x \to y$: *different* decoders from the *same* activities, fit to the step:

  $$\mathbf{d}^{\text{ch}} = \arg\min_{\mathbf{d}} \sum_k \Big(\text{choice}(x_k) - \sum_i d_i a_i(x_k)\Big)^2 + (\text{regularization}),$$

  $$\hat{c}(x) = \sum_i d_i^{\text{ch}} a_i(x) \approx \text{choice}(x).$$

- Population y is fed through its input synapse τ_s (Nengo's default, 5 ms). With no recurrence ($A' = 0$) it is a fast low-pass filter:

$$\tau_s \dot{y} = -y + \hat{c}(x(t)) \quad \Longrightarrow \quad y(t) \approx \hat{c}(x(t)) \text{ lagged by } \approx 5 \text{ ms.}$$

- The output is decoded from y's own neurons: $\hat{y} = \sum_j d_j^y b_j(y)$, with $b_j(y) = G_j[\alpha_j e_j y + J_j^{\text{bias}}]$. With intercepts in $[0.4, 0.9]$, every $b_j = 0$ for $|y| < 0.4$, so $\hat{y} = 0$ exactly until the decision.

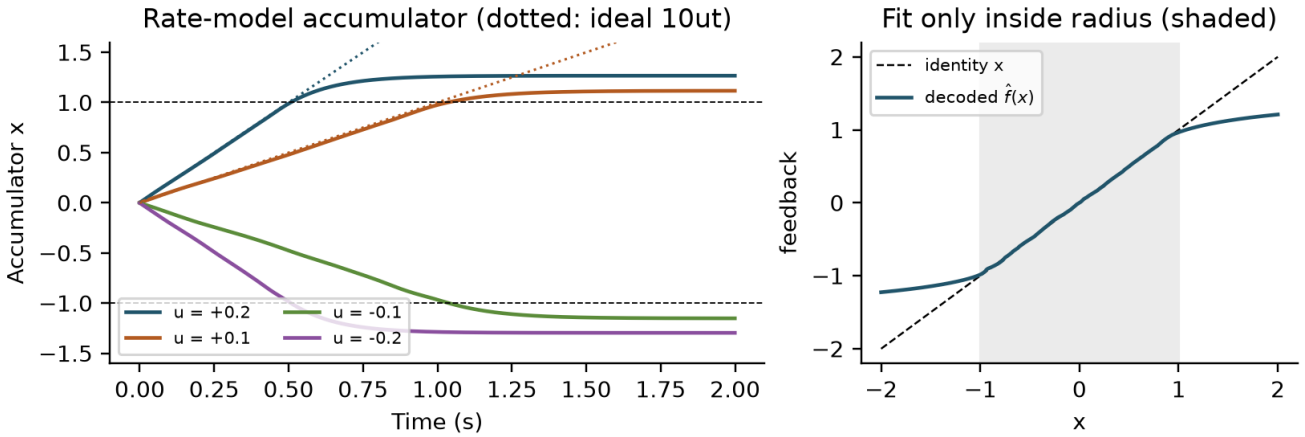
3. With noise (§3.3). Add a random term to the accumulator: $\dot{x} = 10u + \xi(t)$. The decision time is now random, $T = \min\{t : |x(t)| \geq 0.9\}$, and the choice is $\text{sign}(x(T))$, which can be wrong.

Worked example: $u = 0.2$ and $u = -0.1$ (ideal model)

$u = 0.2$: $x = 2t$ reaches 0.9 at $t^* = 0.45$ s; $y = 0$ before, $+1$ after; x then levels off just above 1 (radius).
$u = -0.1$: $x = -t$ reaches -0.9 at $t^* = 0.9$ s; $y = 0$ before, -1 after.
Halving the evidence doubles the decision time, and the sign of u sets the choice.

Combined 2-D version (§3.6). One ensemble represents $\mathbf{v} = (x, y)$ with radius $\geq \sqrt{2}$. The input goes to v_0; a recurrent identity connects $v_0 \to v_0$; a second connection from the ensemble to v_1 computes choice(v_0). The equations are the same, $\tau\dot{x} = -x + \hat{f}(x) + u$ and $\tau\dot{y} = -y + \hat{c}(x)$, but both are decoded from one shared set of neurons.

3.2 Why the value saturates near ± 1



The recurrent connection does not compute x exactly. It computes $\hat{f}(x)$, decoded from neurons whose decoders were fit only on evaluation points inside the radius. Inside $[-1, 1]$, $\hat{f}(x) \approx x$. Outside, nothing constrains the fit, LIF rates flatten as they approach their maximum, and no neurons have intercepts beyond 1 to be recruited. So $|\hat{f}(x)| < |x|$ beyond the radius (right panel).

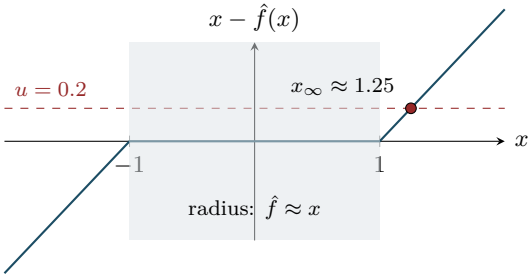
Derivation: where the accumulator stops

With the actual decoded feedback, the boxed L06 equation reads

$$\tau\dot{x} = -x + \hat{f}(x) + u \qquad \Longrightarrow \qquad \dot{x} = 0 \iff x - \hat{f}(x) = u.$$

Inside the radius, $x - \hat{f}(x) \approx 0$, so for $u \neq 0$ there is no fixed point and x keeps ramping. Beyond the radius, $x - \hat{f}(x)$ grows, which acts as a leak. The value stops where that leak exactly balances

the input. Stronger input settles slightly further out.



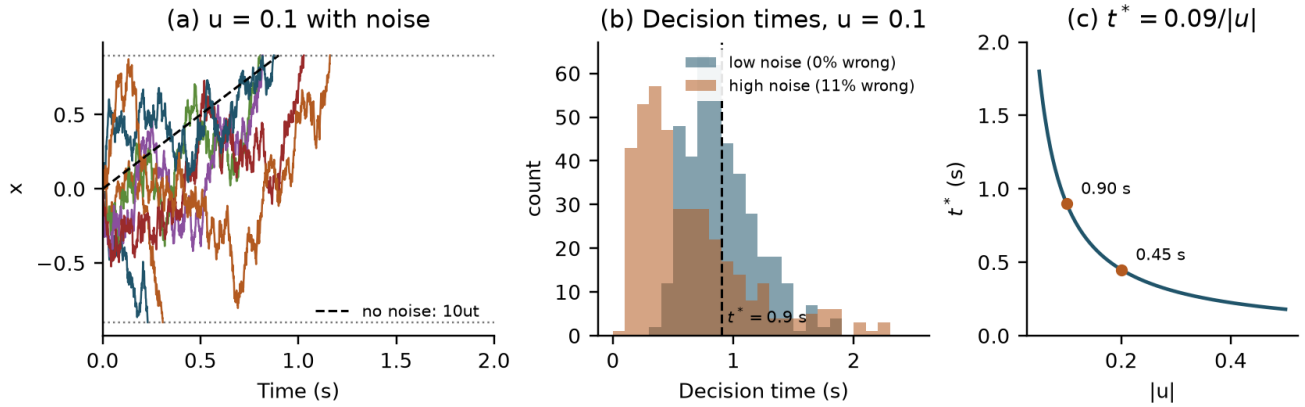
On the test

For "why does it stop near ±1?", name the cause (decoders fit only inside the radius, so $\hat{f}(x) < x$ beyond it), the mechanism (the gap $x - \hat{f}(x)$ acts as a leak) and the observable (the ramp levels off just beyond ±1, a bit further out for stronger u). Saturation is a by-product of the radius, not a designed threshold.

3.3 Adding noise: drift-diffusion

Injecting noise into the ensemble makes the model $\dot{x} = 10u + \xi(t)$: a random walk with drift (the *drift-diffusion model* of decision making). As the noise RMS grows (1, 3, 5, 10):

- traces become jagged around the ideal ramp;
- the time to reach a threshold spreads out into a right-skewed distribution, and with strong noise many decisions come *earlier* than the noise-free t^*;
- weak evidence sometimes reaches the *wrong* bound first, so errors appear;
- at very large noise the value is dominated by noise and choices approach chance.

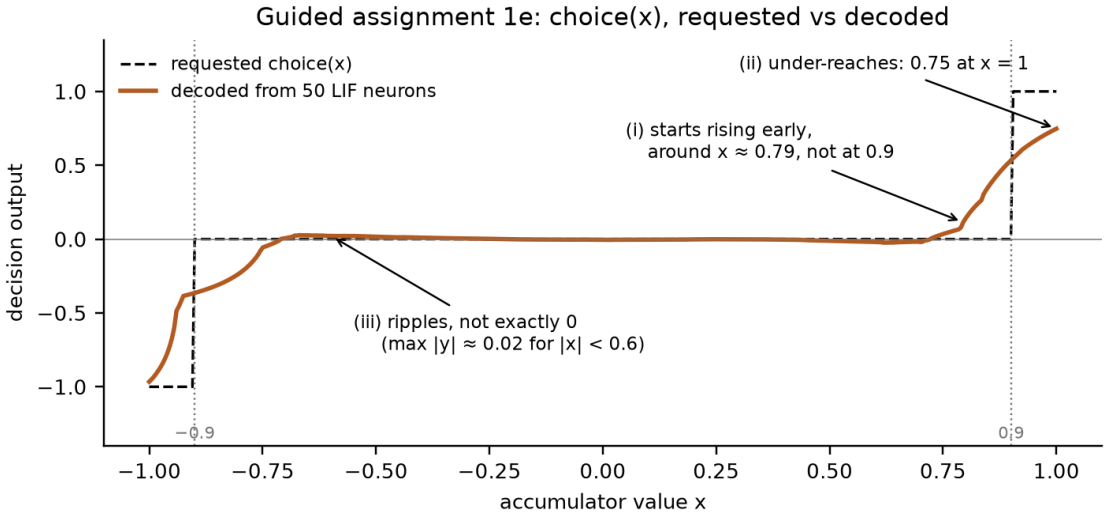


(a) Noisy traces for $u = 0.1$ stop when they hit ±0.9; one hits the wrong bound. (b) Decision times: more noise widens and skews the distribution and produces errors. (c) The noise-free decision time $t^* = 0.09/|u|$ from §3.3: halving the evidence doubles the time.

3.4 Reading out a decision

A second 50-neuron population receives choice(x): +1 if $x > 0.9$, −1 if $x < -0.9$, else 0.

Decision time. The ideal ramp reaches 0.9 at $t^* = 0.9/(10|u|) = 0.09/|u|$: 0.45 s for $|u| = 0.2$ and 0.9 s for $|u| = 0.1$. Decision time is inversely proportional to evidence strength, the behavioural signature of accumulate-to-threshold models.

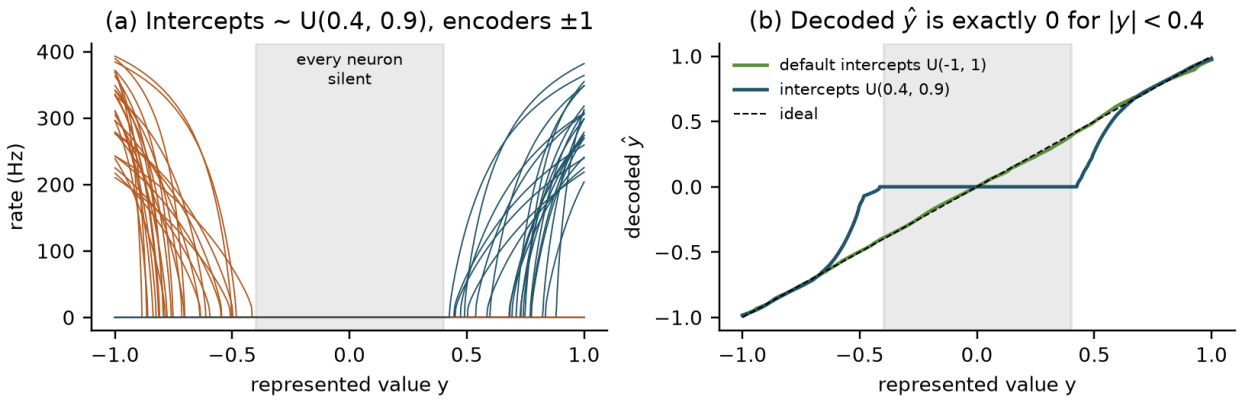


Requested vs actual. The decoded step (i) rises smoothly and early (here from $x \approx 0.79$), (ii) under-reaches ± 1 (here only 0.75 at $x = 1$; how much depends on where the random neurons sit, so the negative side differs), and (iii) has small ripples near zero, so it is never exactly 0. A finite sum of continuous tuning curves is continuous and cannot jump. A sharp step needs high-order components, which have small singular values (Section 7), so the regularized decoders suppress them.

3.5 The intercept trick

Set the output population's intercepts to Uniform$(0.4, 0.9)$ with encoders ± 1:

- **Exactly zero before a decision.** For $|y| < 0.4$ every neuron is silent. The decoder is linear with no constant term, so $\hat{y} = \sum_i d_i \cdot 0 = 0$ *exactly*, not approximately. Small ripples from the choice decoding fall inside this dead zone.
- **Sharper decision.** All neurons are devoted to $|y| > 0.4$, where ± 1 lives, and they switch on steeply.
- **Cost.** Values with $|y| < 0.4$ cannot be represented; they all read as 0. That is acceptable only because the output should be 0 or ± 1.



(a) With intercepts in $[0.4, 0.9]$, no neuron fires for $|y| < 0.4$. (b) The decoded value is a weighted sum of silent neurons there, so it is exactly 0, not just close. The default population (green) represents the whole range but is never exactly 0 except at $y = 0$.

3.6 One 2-D ensemble instead of two 1-D ensembles

Represent (x, y) = (evidence, decision) in one population.

- The radius must be at least $\sqrt{2}$ to reach the corners $(\pm 1, \pm 1)$.
- With 100 neurons accuracy drops (fewer neurons per dimension, a 2-D space to tile, larger radius). With 500 it recovers.
- The intercept trick fails. An intercept is a threshold on $\langle \mathbf{e}, \mathbf{v} \rangle / r$ for the *whole* vector, so high intercepts would silence neurons over most of the space, including small accumulator values. You cannot make a neuron blind near zero in only the decision coordinate.
- **Biological claim.** Separate ensembles claim distinct evidence and decision populations. One ensemble claims the same neurons carry both (*mixed selectivity*). Test it by recording single neurons: do individual cells depend on both accumulated evidence and eventual choice, or split cleanly into two groups? Selective inactivation that impairs one variable but not the other argues for separation.

4 Representing functions and the LMU (Lecture 7, PN4 §2)

4.1 Functions as vectors

Goal. Represent a window of the past, $f_{[t-\theta,t]}(\theta') = u(t - \theta')$ for $0 \le \theta' \le \theta$, as a vector a population can hold. $\theta' = 0$ is "now" and $\theta' = \theta$ is "θ seconds ago." *Representing* a function (the state stands for a whole function) differs from *computing* one (a connection maps x to $f(x)$).

Parametrized families. If f is known to be, say, affine $mt + b$, store $\mathbf{x} = (m, b)$.

Sampling. Store $x_i = f(i\,\Delta t)$. In general this needs infinitely many samples, but:

Key result: Nyquist–Shannon

If f has no frequencies above B, it is *completely* determined by samples spaced $\Delta t = 1/(2B)$ apart: $N = 2BT$ samples for a window of length T. The mapping between samples and function is one-to-one. Storing many more samples than N wastes space.

For example, a 2 Hz signal over a 0.5 s window needs $N = 2$; a 5 Hz signal needs 10 samples per second.

Basis functions. Write $f(t) \approx \sum_i x_i \phi_i(t)$; the vector $\mathbf{x}$ encodes f. Evaluating the represented function at a fixed t is a dot product with $\phi_i(t)$, which is a fixed linear decoder.

Derivation: the coefficients

Take the inner product $\langle \phi_k, \cdot \rangle = \int \phi_k(t) \cdot dt$ of $f = \sum_j x_j \phi_j$:

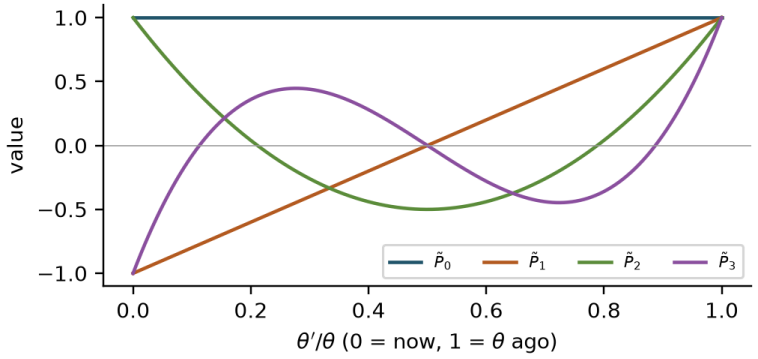
$$\langle \phi_k, f \rangle = \sum_j x_j \langle \phi_k, \phi_j \rangle.$$

If the basis is **orthonormal**, only $j = k$ survives with value 1, so $x_k = \langle \phi_k, f \rangle$ (L07). If it is only **orthogonal**, $x_k = \langle \phi_k, f \rangle / \langle \phi_k, \phi_k \rangle$. In general, solve least squares on samples, which is exactly the decoder problem: *tuning curves are a basis, and decoders are coefficients.* Sampling is the special case $\phi_i = \delta(t - i\Delta t)$, for which $\langle \phi_i, f \rangle = f(i\Delta t)$.

Shifted Legendre polynomials on $r \in [0, 1]$ (orthogonal, not orthonormal; $\|\tilde{P}_i\|^2 = 1/(2i+1)$):

$$\tilde{P}_0 = 1, \quad \tilde{P}_1 = 2r - 1, \quad \tilde{P}_2 = 6r^2 - 6r + 1, \quad \tilde{P}_3 = 20r^3 - 30r^2 + 12r - 1.$$

Values to know: $\tilde{P}_i(0) = (-1)^i$, $\tilde{P}_i(1) = 1$, $\tilde{P}_1(\frac{1}{2}) = \tilde{P}_3(\frac{1}{2}) = 0$, $\tilde{P}_2(\frac{1}{2}) = -\frac{1}{2}$.



Worked example: expanding r and r^2 in the Legendre basis

$r = \frac{1}{2}\tilde{P}_0 + \frac{1}{2}\tilde{P}_1$, since $\frac{1}{2} + \frac{1}{2}(2r-1) = r$.
r^2: from $\tilde{P}_2 = 6r^2 - 6r + 1$, $r^2 = \frac{1}{6}\tilde{P}_2 + r - \frac{1}{6} = \frac{1}{3}\tilde{P}_0 + \frac{1}{2}\tilde{P}_1 + \frac{1}{6}\tilde{P}_2$. Check: $\frac{1}{3} + \frac{1}{2}(2r-1) + \frac{1}{6}(6r^2 - 6r + 1) = r^2$.
An order-k polynomial needs only $\tilde{P}_0, \ldots, \tilde{P}_k$.

4.2 A delay as a dynamical system

Why a delay stores the whole window. A system that outputs $u(t - \theta)$ must eventually output every value currently inside $[t - \theta, t]$, because each one will reach the end of the window. So information about the whole window must be in its state.

In the Laplace domain a perfect delay is $e^{-s\theta}$, which is not a finite-dimensional system. Padé approximants of order q give the closest q-dimensional LTI system:

Key result: LMU / delay network (L07)

$$\dot{\mathbf{x}} = A\mathbf{x} + Bu, \qquad \theta A_{ij} = (2i+1) \begin{cases} -1, & i < j, \\ (-1)^{i-j+1}, & i \geq j, \end{cases} \qquad \theta B_i = (2i+1)(-1)^i, \qquad i, j = 0, \ldots, q-1.$$

Implement it with $A' = \tau A + I$ and $B' = \tau B$. Decode the past with

$$u(t-\theta') \approx \sum_{i=0}^{q-1} \tilde{P}_i\left(\frac{\theta'}{\theta}\right) x_i(t).$$

Derivation: the $q = 1$ LMU is a low-pass filter

For $q = 1$ only $i = j = 0$: $\theta A_{00} = 1 \cdot (-1)^1 = -1$ and $\theta B_0 = 1$. So

$$\dot{x} = \frac{u - x}{\theta}.$$

Laplace: $sX = (U - X)/\theta \Rightarrow X/U = 1/(1 + \theta s)$. Compare the delay: $e^{-s\theta} = 1/e^{s\theta} \approx 1/(1 + s\theta)$, the first-order expansion of the denominator. That is the [0/1] Padé approximant.

Meaning. x is an exponentially weighted average of roughly the last θ seconds. With $q = 1$ the decoder is $\tilde{P}_0 = 1$, so the "delayed" output is just x: a smeared, early version of the delay. Its step response $1 - e^{-t/\theta}$ is already 0.63 at $t = \theta$, whereas a true delay stays at 0 until $t = \theta$ and then jumps.

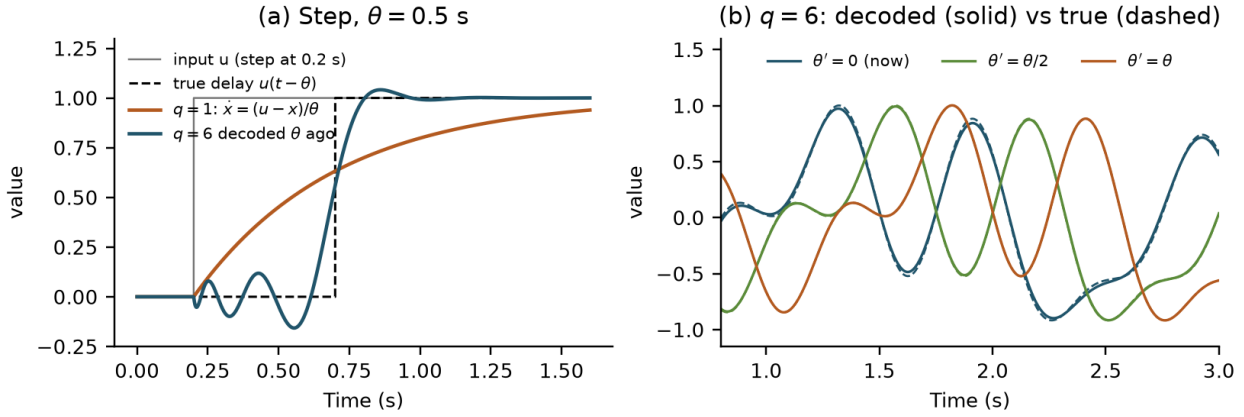
Neural transforms ($\tau = 0.1$, $\theta = 0.5$): $A' = 1 - \tau/\theta = 0.8$, $B' = \tau/\theta = 0.2$.

Worked example: $q = 2$

$\theta A = \begin{pmatrix} -1 & -1 \\ 3 & -3 \end{pmatrix}$ (i=0, j=1 has $i < j$ so -1; i=1, j=0 gives $3(-1)^2 = 3$; i=j=1 gives $3(-1)^1 = -3$), and $\theta B = \begin{pmatrix} 1 \\ -3 \end{pmatrix}$. With $\tau/\theta = 0.2$: $A' = 0.2\,\theta A + I = \begin{pmatrix} 0.8 & -0.2 \\ 0.6 & 0.4 \end{pmatrix}$ and $B' = \begin{pmatrix} 0.2 \\ -0.6 \end{pmatrix}$. In general A' is $q \times q$ and B' is $q \times 1$.

Worked example: decoding weights for $q = 4$

Now ($\theta' = 0$): weights $\tilde{P}_i(0) = (1, -1, 1, -1)$, so $u(t) \approx x_0 - x_1 + x_2 - x_3$.
θ ago ($\theta' = \theta$): weights $(1, 1, 1, 1)$, so $u(t - \theta) \approx x_0 + x_1 + x_2 + x_3$.
Middle ($\theta' = \theta/2$): weights $(1, 0, -\frac{1}{2}, 0)$.
Each delay is a different *fixed linear decoder* from the same state. Any function of the window, not only delays, can be decoded by a connection from the ensemble.



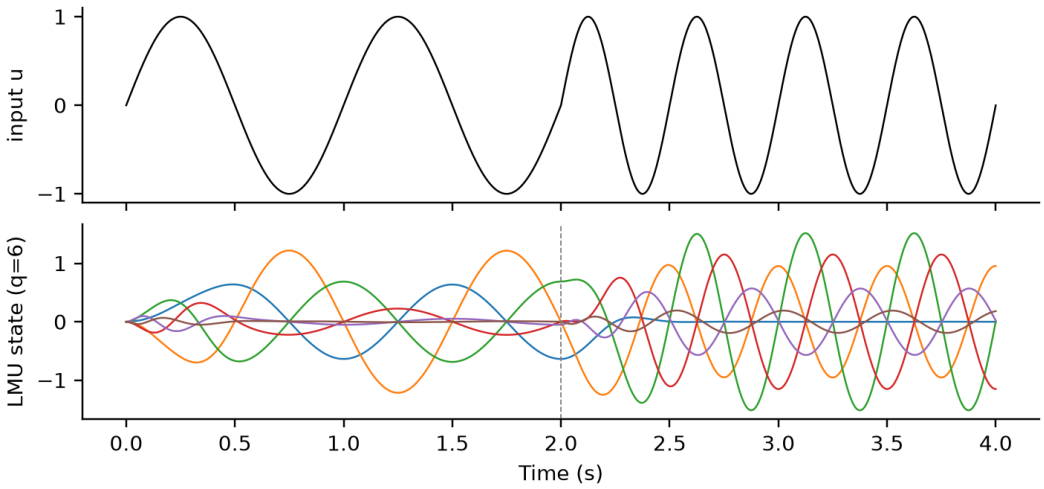
Ideal (non-neural) LMU, $\theta = 0.5$ s. (a) The $q = 1$ LMU (orange) starts rising immediately: it is a low-pass filter, not a delay. With $q = 6$, decoding with weights $\tilde{P}_i(1) = 1$ gives a step at the right time, with some ringing. (b) The same $q = 6$ state decoded with three different weight vectors gives three different delays of a band-limited input.

Trap: index ranges and the decoding direction

The L07 bullet writes $\sum_{i=1}^{q}$, but the matrices and code index from 0 to $q-1$. Use $0, \ldots, q-1$. The convention $\tilde{P}_i(1) = 1$ decodes θ *ago* and $\tilde{P}_i(0) = (-1)^i$ decodes *now*; this was checked numerically. Because the basis is orthogonal but not orthonormal, $x_i = (2i+1)\int_0^1 f(r)\tilde{P}_i(r)\,dr$, so x_0 is the plain window average.

Implementation details. The LMU ensemble is q-dimensional. L07 draws its intercepts from `CosineSimilarity(q+2)`: in high dimensions $\langle \mathbf{e}, \mathbf{x} \rangle$ for random unit vectors concentrates near 0, so uniform intercepts would leave many neurons always on or always off. The state components can exceed 1 for a unit-amplitude input, so scale the input or radius to avoid the saturation of Section 3.

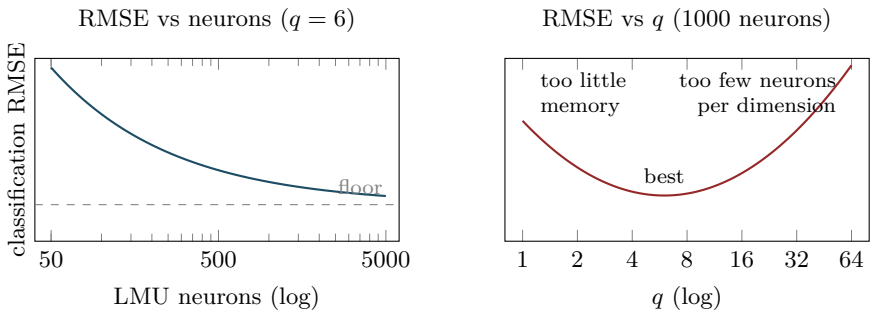
4.3 Classifying 1 Hz vs 2 Hz (PN4 §2)



Why the current value is not enough. Both sines pass through every value in $[-1, 1]$, so $u(t)$ alone is ambiguous. Frequency is a property of the signal over time.

Why the window works. With $\theta = 0.5$ s the window holds half a cycle of 1 Hz but a full cycle of 2 Hz. So x_0, the window average, is about 0 for 2 Hz but oscillates with amplitude about $2/\pi \approx 0.64$ for 1 Hz. The 2 Hz energy moves into higher-order coefficients. The two classes trace different trajectories in the 6-D state, and a decoder can separate them. Note the change at $t = 2$ s in the figure.

Swapping the order (2 Hz first). The first ≈ 0.25 s is misclassified. At start-up the window is mostly empty, so the state is near zero and carries little frequency information, and the classifier only ever saw start-up states labelled +1. Fixes: train on both orders, exclude the first θ seconds from training, or allow an "undecided" output.



Qualitative shapes to sketch (not simulated values).

RMSE vs neurons: decreases steeply from 50 neurons (far too few for a 6-D state), with diminishing returns (noise error scales roughly as $1/\sqrt{n}$), levelling off at a floor set by the classifier and finite window.

RMSE vs q (1 to 64, log axis): U-shaped. Small q cannot describe the window's shape, so the classes overlap. Large q spreads a fixed 1000 neurons over many dimensions, and high-order coefficients have large gains $(2i+1)/\theta$ and fast dynamics that 0.1 s synapses and a finite radius cannot implement. The best q is moderate.

5 Supervised learning (Lecture 8, PN4 §3–4)

5.1 What "learning" means here

Learning is a directed change in synaptic weights $\mathbf{W}$ while the network is active. Any past-affects-future phenomenon (integrators, the delay network, firing-rate adaptation) is **adaptation**, which includes learning.

Why learn when we can solve for decoders? (1) The function may not be known in advance (which food is edible?). (2) The required function may change (injury, growth, new loads). (3) Offline decoders are optimal for $\mathbf{D}$ and static rates, not for the full $\mathbf{W} = \mathbf{ED}$ or the spiking dynamics. (4) To answer scientific questions about how brains learn.

Types (ML). Supervised: pairs $(\mathbf{x}_k, \mathbf{t}_k)$, fit $f(\mathbf{x}; \mathbf{w}) \approx \mathbf{t}$. Unsupervised: only $\mathbf{x}_k$, find structure or a low-dimensional latent $\boldsymbol{\lambda}$. Reinforcement: states and a scalar reward, learn a policy. Separately: offline, online, or life-long.

5.2 Gradient descent and the delta rule

Gradient descent nudges parameters against the gradient of a loss: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E(\mathbf{w})$, with learning rate η. Using one sample at a time is stochastic gradient descent with batch size 1.

Derivation: the delta rule for decoders (L08)

Scalar case. The population represents $x(t)$; we want $\hat{y} = \sum_i d_i a_i(x)$ to track $y^{\mathrm{d}}(t)$. The instantaneous loss is

$$E(\mathbf{d}) = \frac{1}{2}\Big(\sum_i d_i a_i(x(t)) - y^{\mathrm{d}}(t)\Big)^2.$$

Chain rule: the outer derivative is the bracket and the inner derivative of $\sum_i d_i a_i$ with respect to d_j is a_j:

$$\frac{\partial E}{\partial d_j} = \underbrace{\Big(\sum_i d_i a_i - y^{\mathrm{d}}\Big)}_{\epsilon(t)} a_j = \epsilon\, a_j \qquad \Longrightarrow \qquad \boxed{d_j \leftarrow d_j - \eta\, \epsilon(t)\, a_j(t)}, \quad \epsilon = \hat{y} - y^{\mathrm{d}}.$$

Meaning. If neuron j is active while the output is too high ($\epsilon > 0$), its decoder decreases, so it pushes the output up less. If the output is too low, its decoder increases. Silent neurons do not change.

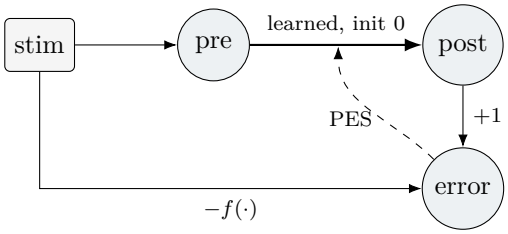
5.3 From decoders to weights: PES

Decoders do not exist in the brain; weights do. Use $w_{ij} = \alpha_i \langle \mathbf{e}_i, \mathbf{d}_j \rangle$ and substitute the decoder update:

Key result: Prescribed Error Sensitivity (PES)

$$\Delta w_{ij} = \alpha_i \langle \mathbf{e}_i, \Delta \mathbf{d}_j \rangle = -\eta \underbrace{\alpha_i \langle \boldsymbol{\epsilon}, \mathbf{e}_i \rangle}_{\text{modulatory, post}} \underbrace{a_j}_{\text{pre}} \qquad \text{(scalar: } -\eta\, \alpha_i e_i\, \epsilon\, a_j\text{)}.$$

The term $\alpha_i \langle \boldsymbol{\epsilon}, \mathbf{e}_i \rangle$ is the error encoded into neuron i *as if it were an input current*: a **modulatory** input, like dopaminergic synapses that enable plasticity. The rule is **local**: each synapse needs its presynaptic activity and a signal delivered to its postsynaptic neuron.



error $= \hat{y} - y^{\mathrm{d}}$ = output − target. For the identity, $f(x) = x$.

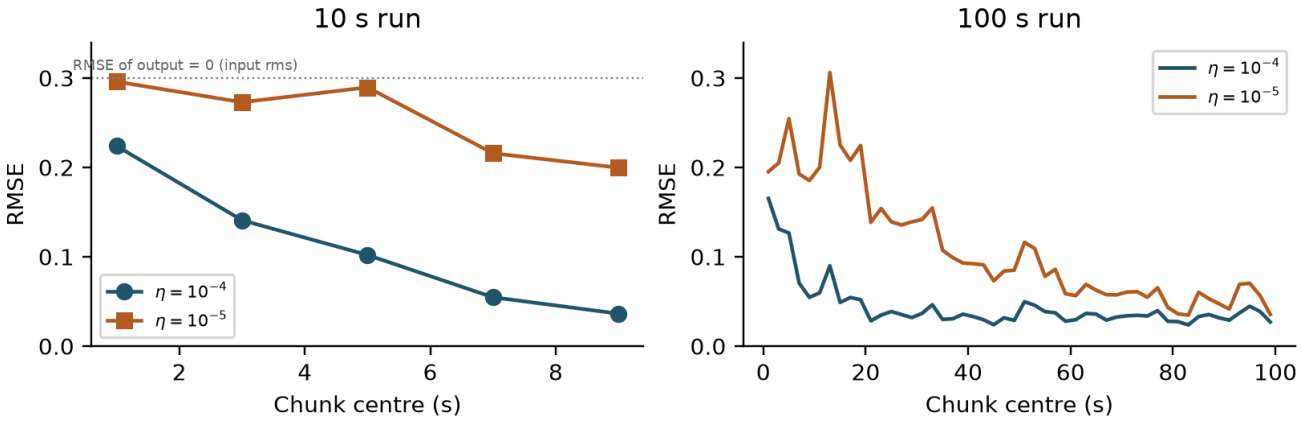
Trap: error sign convention

The L08 derivation uses $\epsilon = \hat{y} - y^{\mathrm{d}}$ with $\Delta d = -\eta \epsilon a$, and so does PN4 ("output − desired"). One L08 figure caption writes $\epsilon = y^{\mathrm{d}} - y$. Either works if the update sign matches. Say which you use.

Worked example: reversing the error sign (Practice Test Q3b)

Computing $y^{\mathrm{d}} - \hat{y}$ with the same update makes the rule climb the loss: *gradient ascent*. An output that is too high is pushed higher, which is positive feedback, so the output runs away from the target until the populations saturate and the error grows.

5.4 Learning curves and limits



Chunked RMSE (2 s chunks, averaged over runs): falls across chunks and levels off *above* zero. With $\eta = 10^{-5}$, learning is about 10× slower; early progress scales roughly with $\eta\times$ time. Over 100 s the slow rate approaches the floor, and can end slightly quieter because the weights jitter less.

Why the error never reaches zero: finite neurons (static distortion), noisy spiking pre and error signals, weight jitter from ongoing updates, and filter lag.

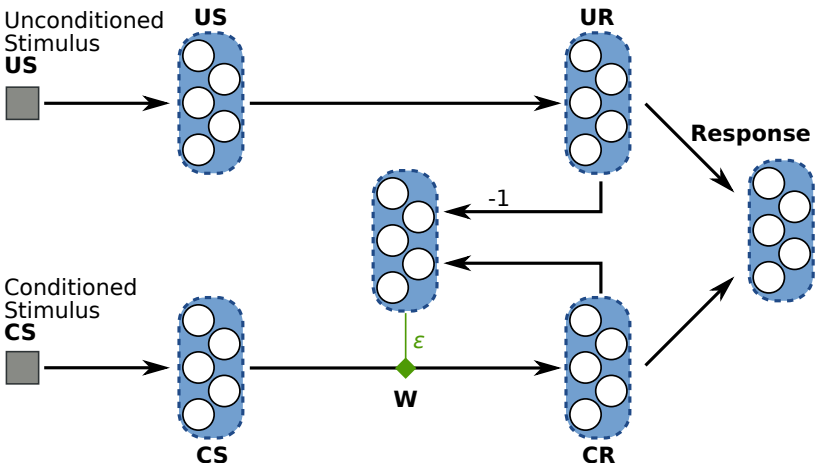
To improve: more neurons, a smaller or annealed η with longer training, inputs covering the whole domain, a filtered error, or starting from offline decoders. With enough training, PES can beat offline decoders because it learns from the actual spikes rather than the rate approximation.

Function difficulty. With the same η, $f(x) = x$ is learned much faster than x^2. Section 7.5 derives why.

Catastrophic forgetting. PES only uses the current sample. If training stays in one region (say $x \in [0.5, 1]$), the neurons active there change, and many of them also fire elsewhere, so the mapping elsewhere drifts with no error to correct it. The network over-fits the current region.

Vector outputs. To learn $[x^2, -x]$, make post and error 2-D, keep pre 1-D, and put the function on the stim→error connection with a minus sign.

5.5 Classical conditioning



An unconditioned stimulus (US) causes an unconditioned response (UR). Pairing a neutral conditioned stimulus (CS) with the US makes the CS alone produce a conditioned response (CR). In the PES model, the UR acts as the teaching signal: the error compares the response predicted from the CS with the UR, so the error is generated *inside* the system and no external teacher is needed. **Extinction:** the CS appears without the US, the target is zero, and the learned CS→response mapping weakens.

5.6 Adaptive control of a pendulum (PN4 §4)

Angle θ from horizontal, counter-clockwise positive:

$$mL^2\ddot{\theta} = \tau - mgL\cos\theta, \qquad \tau_{PD} = k_p(\theta^* - \theta) - k_d\dot{\theta}.$$

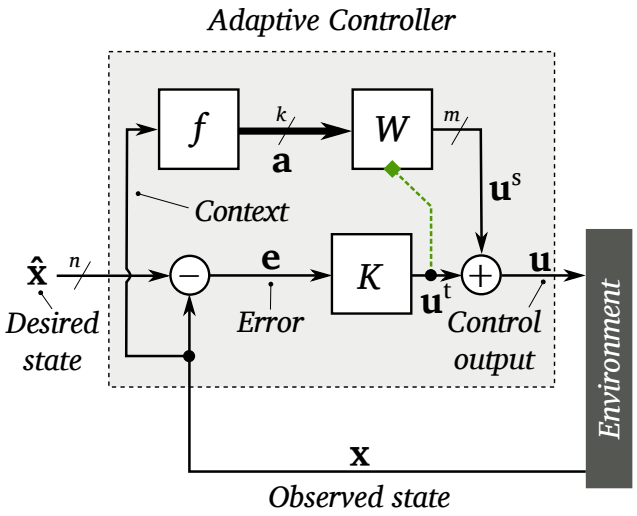
Gravity's torque $mgL\cos\theta$ is largest at $\theta = 0$ (horizontal) and zero at $\pm\pi/2$ (vertical).

Derivation: PD steady-state offset

At rest, $\ddot{\theta} = \dot{\theta} = 0$, so the torques balance: $k_p(\theta^* - \theta) = mgL\cos\theta$. Write $\theta = \theta^* + \delta$ and assume δ is small, so $\cos\theta \approx \cos\theta^*$:

$$\delta \approx -\frac{mgL\cos\theta^*}{k_p} \qquad \Longrightarrow \qquad \theta_{ss} \approx \theta^* - \frac{mgL\cos\theta^*}{k_p}.$$

With $m = L = 1$, $g = 9.8$, $\theta^* = 0$, $k_p = 49$: $\theta_{ss} \approx -0.2$ rad ($\approx -11°$), *below* the target, because gravity pulls down. At $\theta^* = \pi/2$ there is no offset. Raising k_p shrinks the offset only as $1/k_p$, amplifies noise, demands large torques and still cannot adapt to a changed load.



Derivation: what the adaptive ensemble learns, and why the PD output is the error

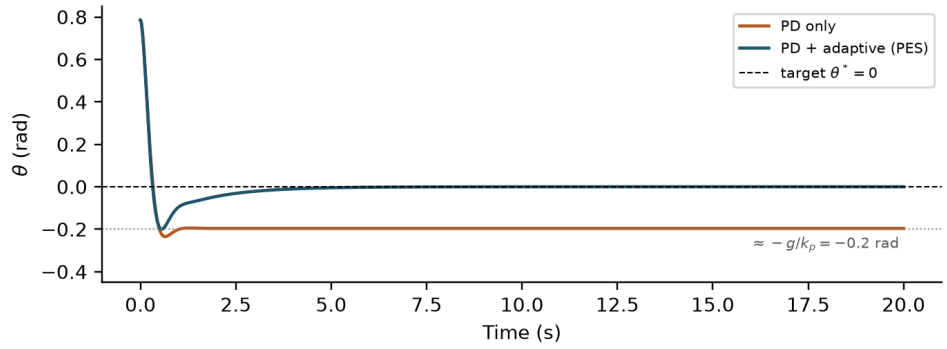
Add an adaptive torque u_{ad} decoded from an ensemble representing the state, through a PES connection initialized to zero. Total torque $\tau = \tau_{PD} + u_{ad}$. At equilibrium:

$$0 = \tau_{PD} + u_{ad} - mgL\cos\theta \qquad \Longrightarrow \qquad \tau_{PD} = mgL\cos\theta - u_{ad}.$$

So the PD output equals exactly the part of gravity that the adaptive term is still missing. Learning that drives $\tau_{PD} \to 0$ drives $u_{ad} \to mgL\cos\theta$, and the offset vanishes. Nothing needs to supply the correct torque.

Sign. When $\tau_{PD} > 0$, the adaptive torque is too small and should increase. With $\Delta d = -\eta\epsilon a$, an increase needs $\epsilon < 0$, so $\epsilon = -\tau_{PD}$: connect the PD signal to the learning rule with transform -1.

Why represent the state? The required torque depends on θ. A constant input could only learn one torque value, correct at one angle.



Sketch: both start at $\pi/4 \approx 0.79$. PD alone settles at -0.2. With adaptation, the same transient, then the offset decays to 0 over several seconds. Same mechanism as §5.3 (PES on decoders from an ensemble, driven by an error), but there is no explicit target function: the controller generates the error, and the learned output acts through the plant. The learned function is accurate mainly near states visited during training.

6 Unsupervised learning (Lecture 8)

Goal. Given only samples $\mathbf{x}_k$, find a low-dimensional latent $\boldsymbol{\lambda}$ that describes them (for example, a few dozen face features rather than thousands of pixels). **Autoencoders** train $f(\mathbf{x}) \approx \mathbf{x}$ through a narrow bottleneck; they are powerful but rely on back-propagation, which is biologically implausible.

6.1 Principal component analysis

A basis transformation is an orthogonal $\mathbf{T}$ ($\mathbf{T}\mathbf{T}^T = \mathbf{I}$): $\boldsymbol{\lambda} = \mathbf{T}\mathbf{x}$ and $\mathbf{x} = \mathbf{T}^T\boldsymbol{\lambda}$, which is lossless. PCA picks $\mathbf{T}$ so that the first component has maximum variance, the next has maximum variance of what remains, and so on. Keeping only the first d' components compresses the data, and the components may be interpretable (eigenfaces).

Key result: PCA

The principal directions are the eigenvectors of the covariance matrix $\mathbf{C} = \frac{(\mathbf{X}-\bar{\mathbf{x}})^T(\mathbf{X}-\bar{\mathbf{x}})}{N-1} = \mathbf{V}\Lambda\mathbf{V}^T$, sorted by eigenvalue.

Derivation: PCA by SVD

Let $\mathbf{X} - \bar{\mathbf{x}} = \mathbf{U}\mathbf{S}\mathbf{V}^T$. Then

$$\mathbf{C} = \frac{\mathbf{V}\mathbf{S}\mathbf{U}^T\mathbf{U}\mathbf{S}\mathbf{V}^T}{N-1} = \mathbf{V}\frac{\mathbf{S}^2}{N-1}\mathbf{V}^T,$$

using $\mathbf{U}^T\mathbf{U} = \mathbf{I}$. So the right singular vectors $\mathbf{V}$ are the principal directions and the eigenvalues are $S^2/(N-1)$. No covariance matrix is needed.

Limitation: PCA is linear; it misses curved structure, and the highest-variance direction need not be the useful one (in the face data it is skin tone, which is a warning about bias).

6.2 Hebbian rules

Key result: the rules (post i, pre j)

Hebb	$\Delta w_{ij} = \eta\, a_i a_j$ ("fire together, wire together")
Normalized Hebb	$\Delta w_{ij} = \eta a_i a_j$, then $\mathbf{w}_i \leftarrow (\mathbf{w}_i + \Delta\mathbf{w}_i)/\lVert\mathbf{w}_i + \Delta\mathbf{w}_i\rVert$
Oja	$\Delta w_{ij} = \eta\,(a_i a_j - a_i^2 w_{ij})$
BCM (L08 form)	$\Delta w_{ij} = \eta\, a_j a_i(\theta - a_i)$, θ a target rate

Hebb is unstable. Rates are non-negative, so $\Delta w \geq 0$ always. Larger weights produce more post activity and larger updates, so the weights run away. It also ignores causation: it counts coincident firing, not which neuron caused which.

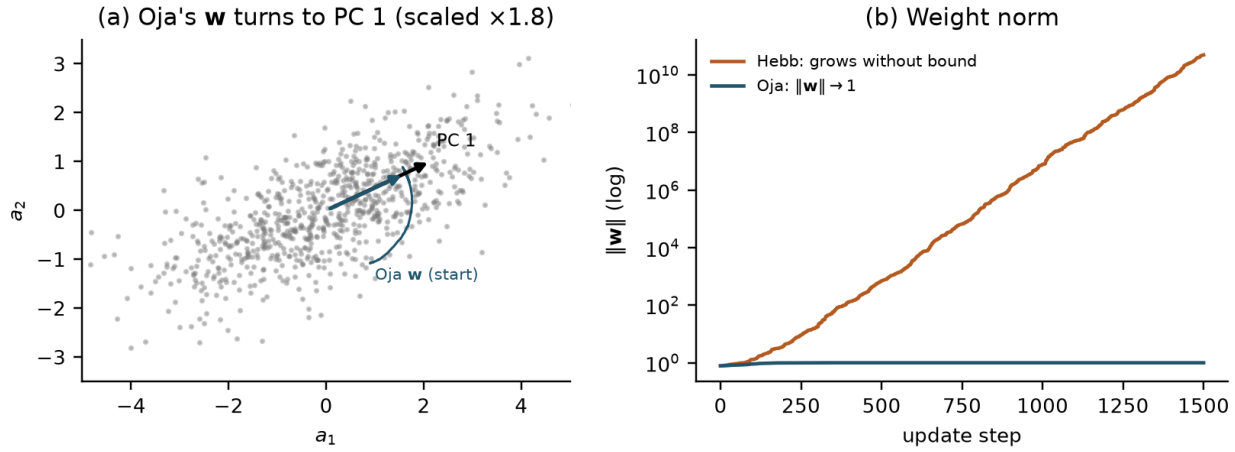
Derivation: Oja's rule from normalized Hebb (linear neuron)

Let $a_i = \sum_j w_{ij} a_j = \mathbf{w}^T\mathbf{a}$ with $\lVert\mathbf{w}\rVert = 1$. To first order in η,

$$\lVert\mathbf{w} + \eta a_i \mathbf{a}\rVert^2 = 1 + 2\eta a_i\, \mathbf{w}^T\mathbf{a} + O(\eta^2) = 1 + 2\eta a_i^2 + O(\eta^2), \quad \text{so} \quad \lVert\mathbf{w} + \eta a_i \mathbf{a}\rVert^{-1} \approx 1 - \eta a_i^2.$$

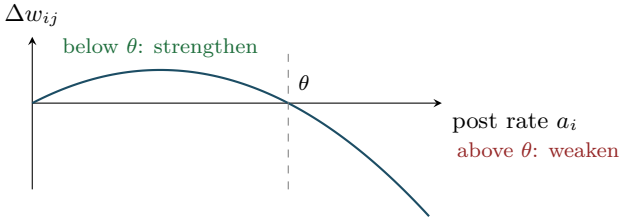
Then $\mathbf{w}_{\text{new}} \approx (\mathbf{w} + \eta a_i \mathbf{a})(1 - \eta a_i^2) \approx \mathbf{w} + \eta(a_i\mathbf{a} - a_i^2\mathbf{w})$, which is Oja's rule: the decay term $-a_i^2 w_{ij}$ replaces the explicit normalization.

Where it converges. Averaged over inputs, $\langle\Delta\mathbf{w}\rangle \propto \mathbf{C}\mathbf{w} - (\mathbf{w}^T\mathbf{C}\mathbf{w})\mathbf{w}$, with $\mathbf{C} = \left\langle \mathbf{a}\mathbf{a}^T \right\rangle$. This is zero only when $\mathbf{w}$ is a unit eigenvector of $\mathbf{C}$, and only the top one is stable: $\mathbf{w} \to$ the first principal component.



A linear neuron with two correlated, zero-mean inputs. (a) Oja's weight vector rotates from its start onto the first principal component (black). (b) Plain Hebb's weight norm grows exponentially, while Oja's settles at 1.

BCM increases weights when the post rate is below the target θ and decreases them above it, a homeostatic rule. Often θ tracks the neuron's recent average rate.



L08's BCM for a fixed active pre-neuron: $\Delta w \propto a_i(\theta - a_i)$ pushes the post rate toward θ.

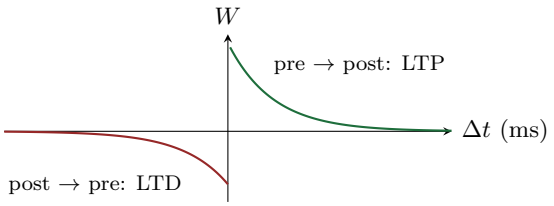
Trap: BCM sign

Standard BCM papers write $\eta a_j a_i(a_i - \theta)$ with a sliding threshold, the opposite sign. Use the lecture's form on the test and say so.

6.3 Spike-timing-dependent plasticity

Hebb's quote says the pre-neuron must *take part in firing* the post-neuron, which implies causality. STDP sums a window over all pre/post spike pairs, with $\Delta t = t^{\text{post}} - t^{\text{pre}}$:

$$\Delta w_{ij} = \eta \sum_{k,\ell} W(t_k^{\text{post}} - t_\ell^{\text{pre}}), \qquad W(\Delta t) = \begin{cases} \alpha^+ e^{-\Delta t/\tau^+}, & \Delta t > 0 \text{ (pre before post: potentiate)}, \\ -\alpha^- e^{+\Delta t/\tau^-}, & \Delta t \le 0 \text{ (post before pre: depress)}. \end{cases}$$



Both sides decay with $|\Delta t|$ over about 10 ms. STDP and BCM may describe the same phenomena when "activity" is read as filtered spikes.

Trap: STDP typo in the notes

L08 prints $-\alpha^- e^{-\theta/\tau^-}$ for $\theta \le 0$ (using θ for Δt). That would grow without bound as $\Delta t \to -\infty$. The exponent should be $+\Delta t/\tau^-$, as above.

7 Analysing representation with the SVD (Lecture 9, PN4 §5)

Observation. Some functions decode with much larger error than others. **Goal:** understand which functions a population supports, independent of any one target.

7.1 Which matrix, and why

From Section 1, $\mathbf{D} = \gamma^{-1}\Upsilon$ with $\gamma = \mathbf{A}^T\mathbf{A}$ and $\Upsilon = \mathbf{A}^T\mathbf{X}$. Every target function changes only Υ; γ **is shared by every decoding**, so its structure sets which functions can be decoded well. That is why we decompose γ, not the decoders for one function. γ is like the correlation matrix of the tuning curves, and similar tuning curves make it singular or nearly singular.

Key result: SVD of γ

γ is square and symmetric, so $\gamma = \mathbf{U}\mathbf{S}\mathbf{U}^T$ with $\mathbf{U}$ orthogonal and $\mathbf{S}$ diagonal (singular values S_m, sorted). A large S_m means:

- large variance of population activity along direction $\mathbf{U}_m$;
- a large error if that direction were left out;
- more independent information about x, robust to noise.

The pseudo-inverse sets $1/S_m \to 0$ when $S_m = 0$, which gives the minimum-norm decoders.

Derivation: rotating into the χ, Φ basis

Decoders. $\gamma^{-1} = (\mathbf{USU}^T)^{-1} = \mathbf{US}^{-1}\mathbf{U}^T$ because $\mathbf{U}^{-1} = \mathbf{U}^T$. So $\mathbf{D} = \mathbf{US}^{-1}\mathbf{U}^T\mathbf{A}^T\mathbf{X}$.

Estimate. Insert $\mathbf{UU}^T = \mathbf{I}$:

$$\hat{\mathbf{X}} = \mathbf{AD} = (\mathbf{AU})(\mathbf{S}^{-1}\mathbf{U}^T\mathbf{A}^T\mathbf{X}) = \chi\Phi, \qquad \chi = \mathbf{AU}, \qquad \Phi = \mathbf{U}^T\mathbf{D} = \mathbf{S}^{-1}\mathbf{U}^T\mathbf{A}^T\mathbf{X}.$$

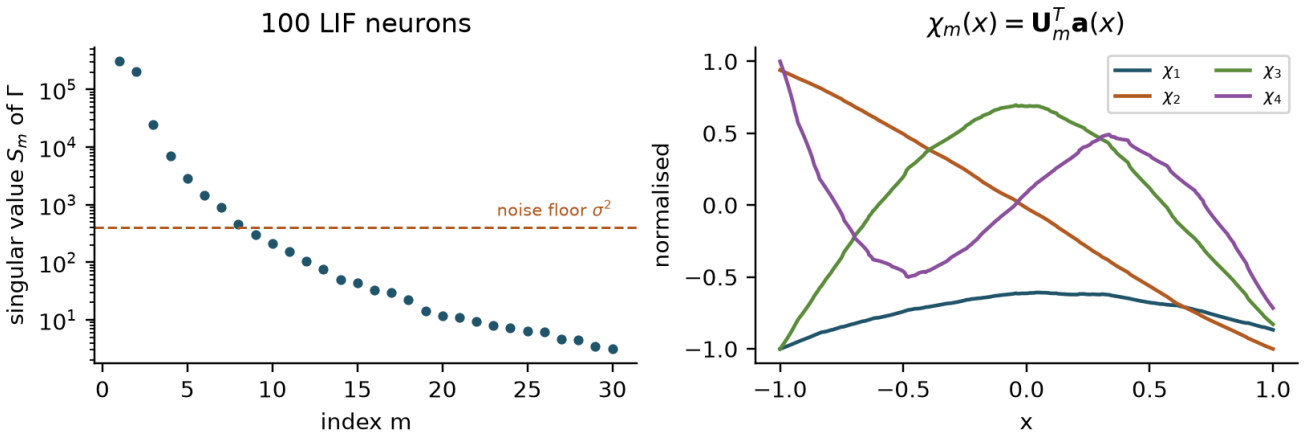
χ and Φ are $\mathbf{A}$ and $\mathbf{D}$ rotated into the coordinates defined by $\mathbf{U}$.

The χ are orthogonal with norms S_m:

$$\chi^T\chi = \mathbf{U}^T\mathbf{A}^T\mathbf{AU} = \mathbf{U}^T(\mathbf{USU}^T)\mathbf{U} = \mathbf{S}, \qquad \text{so} \quad \langle\chi_m\chi_n\rangle_x = S_m\delta_{mn}.$$

Column m of χ is a function of x: $\chi_m(x) = \sum_i U_{im}\, a_i(x)$. These are new, ordered, orthogonal basis functions built from the tuning curves.

7.2 What the basis looks like



Spectrum. The singular values decay quickly (roughly exponentially), then flatten into a tail.

Basis functions. For heterogeneous 1-D LIF populations, $\chi_1, \chi_2, \chi_3, \chi_4$ look like constant, linear, quadratic and cubic **Legendre polynomials** (signs are arbitrary). The population supports functions roughly in polynomial order: low-order functions decode well, and precise high-order structure is error-prone. Broad, roughly linear tuning curves naturally give a broad, ordered polynomial basis. Gaussian (bump) tuning curves give a Fourier-like (sinusoidal) basis instead.

Heterogeneity matters. If intercepts were clumped (on-neurons near $x_{\max}$, off-neurons near $x_{\min}$), the linear term would no longer be better supported than the quadratic, and the ordering breaks. Heterogeneity tiles the space, keeps the low-order singular values large and robust to noise, and is cheap to build.

7.3 Vector representations and cross terms

For D-dimensional $\mathbf{x}$, a function expands into terms like x_1x_2 as well as powers. All terms of order l have singular values of similar size, and the size decays exponentially with l. The number of order-l terms is the number of l-multisets from D variables:

$$N_S(l, D) = \frac{(l + D - 1)!}{l!\,(D - 1)!} = \binom{l + D - 1}{l}.$$

Worked example: counting terms

$D = 2$: order 0: 1; order 1: x_1, x_2; order 2: x_1^2, x_1x_2, x_2^2; order 3: four terms. That is $l+1$ at each order. $D = 3$, $l = 2$: $\binom{4}{2} = 6$ $(x^2, y^2, z^2, xy, xz, yz)$. Higher dimensions have more terms per order and smaller singular values, so high-order functions of vectors need many more neurons.

7.4 Noise and the usable modes

Each neuron's activity carries independent noise η_i with variance σ^2. Noise is isotropic in neuron space and does not shrink with S_m, so small-S_m directions are swamped first.

Trap: η means two things

L09 writes the neuron noise as η_i with variance σ_η^2. L08 uses η for the learning rate. Below, σ^2 is the noise variance. Compare S_m with σ^2 only when γ is normalized the same way (for example, averaged over samples).

Derivation: the error with noise, mode by mode

Step 1 (add noise). $\hat{x} = \sum_i (a_i + \eta_i) d_i$. The noise has zero mean and is independent across neurons, so the cross term vanishes and only the diagonal of $\langle \eta_i \eta_j \rangle = \sigma^2 \delta_{ij}$ survives:

$$E = \left\langle \left(x - \sum_i a_i d_i \right)^2 \right\rangle_x + \sigma^2 \sum_i d_i^2.$$

Step 2 (rotate). With $\mathbf{U}\mathbf{U}^T = \mathbf{I}$: $\sum_i a_i d_i = \mathbf{a}^T \mathbf{U}\mathbf{U}^T \mathbf{d} = \sum_m \chi_m \Phi_m$ and $\sum_i d_i^2 = \mathbf{d}^T \mathbf{U}\mathbf{U}^T \mathbf{d} = \sum_m \Phi_m^2$. So

$$E = \left\langle \left(x - \sum_m \chi_m \Phi_m \right)^2 \right\rangle_x + \sigma^2 \sum_m \Phi_m^2.$$

Step 3 (expand). Let $c_m = \langle \chi_m x \rangle_x$ and use $\langle \chi_m \chi_n \rangle = S_m \delta_{mn}$:

$$E = \left\langle x^2 \right\rangle - 2 \sum_m c_m \Phi_m + \sum_m (S_m + \sigma^2) \Phi_m^2.$$

The modes **decouple**: each contributes $e_m(\Phi_m) = (S_m + \sigma^2)\Phi_m^2 - 2c_m \Phi_m$ on its own.

Step 4 (optimal coefficient, L09). $de_m/d\Phi_m = 2(S_m + \sigma^2)\Phi_m - 2c_m = 0$:

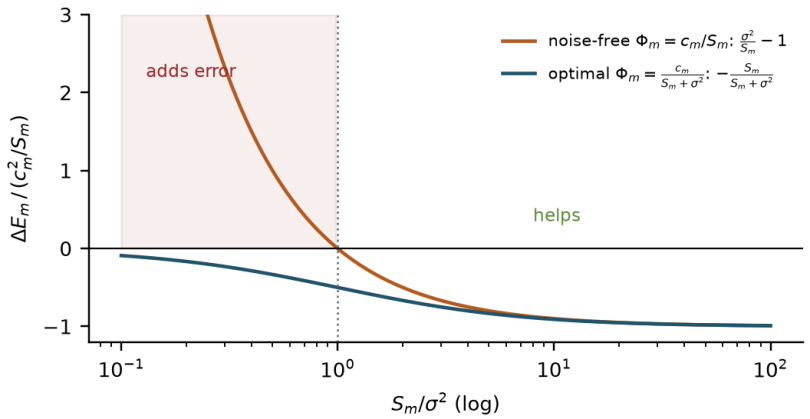
$$\boxed{\Phi_m = \frac{c_m}{S_m + \sigma^2}}, \qquad e_m^{\min} = -\frac{c_m^2}{S_m + \sigma^2}, \qquad E_r = \left\langle x^2 \right\rangle - \sum_m \frac{c_m^2}{S_m + \sigma^2}.$$

If $S_m \gg \sigma^2$, $\Phi_m \approx c_m / S_m$ and noise does not matter. If $S_m \lesssim \sigma^2$, the coefficient shrinks toward 0 and the mode barely helps.

Step 5 (noise-free coefficient, Practice Test Q5c). Including mode m with $\Phi_m = c_m/S_m$ instead of omitting it ($\Phi_m = 0$, $e_m = 0$) changes the error by

$$\Delta E_m = (S_m + \sigma^2)\frac{c_m^2}{S_m^2} - 2\frac{c_m^2}{S_m} = \frac{c_m^2}{S_m} + \frac{\sigma^2 c_m^2}{S_m^2} - \frac{2c_m^2}{S_m} = \frac{c_m^2}{S_m}\left(\frac{\sigma^2}{S_m} - 1 \right).$$

This is negative (it helps) only if $S_m > \sigma^2$, and positive (it *adds* error) if $S_m < \sigma^2$.



Change in error from including mode m, in units of c_m^2/S_m. With the noise-free coefficient (orange), a mode below the noise floor ($S_m < \sigma^2$) makes things worse. With the optimal coefficient (blue), every mode helps a little, but a mode with $S_m \ll \sigma^2$ contributes almost nothing.

Key result: usable modes

Only the basis functions χ_m with $S_m > \sigma^2$ are usable. Functions that need modes below the noise floor cannot be decoded reliably. Dropping those singular values when inverting γ has roughly the same effect as adding noise to γ. Noise sets the representation's effective dimensionality. On a log plot of S_m, draw a horizontal line at σ^2: the modes above it are usable.

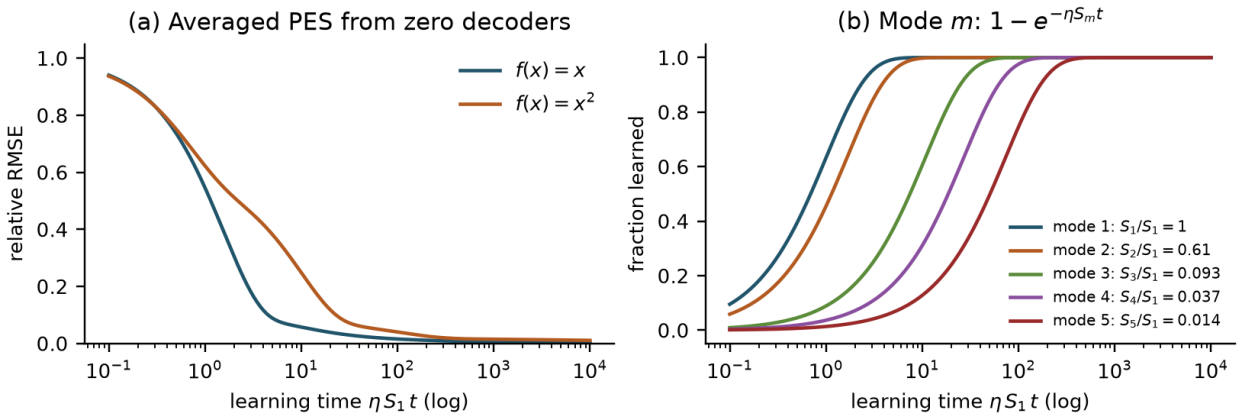
7.5 Connecting the SVD to learning speed

Derivation: why PES learns x faster than x^2

Averaged over inputs, PES is gradient descent on $E(\mathbf{d}) = \frac{1}{2}\left\langle (\mathbf{a}^T\mathbf{d} - y^{\mathrm{d}})^2 \right\rangle$. Its gradient is $\gamma\mathbf{d} - \Upsilon$ (with $\gamma = \left\langle \mathbf{a}\mathbf{a}^T \right\rangle$, $\Upsilon = \left\langle \mathbf{a}\, y^{\mathrm{d}} \right\rangle$), so in continuous time $\dot{\mathbf{d}} = -\eta(\gamma\mathbf{d} - \Upsilon)$. Rotate with $\Phi = \mathbf{U}^T\mathbf{d}$ and $\gamma = \mathbf{U}\mathbf{S}\mathbf{U}^T$:

$$\dot{\Phi}_m = -\eta(S_m\Phi_m - c_m) = -\eta S_m\left(\Phi_m - \frac{c_m}{S_m}\right), \qquad c_m = \left\langle \chi_m y^{\mathrm{d}} \right\rangle.$$

Each mode relaxes to its target independently, with time constant $1/(\eta S_m)$. The identity x lives mainly in the linear mode, which has a large S_m, so it is learned fast. x^2 needs the quadratic mode, which has a smaller S_m, so it is learned more slowly. The same reasoning explains why a decoded step has rounded corners: it needs high-order modes with tiny S_m.



Averaged PES (gradient flow) on a 100-neuron LIF rate population, starting from zero decoders. (a) x^2 takes longer to learn than x. (b) Each mode learns on its own clock $1/(\eta S_m)$; modes 3 and up (quadratic and higher) are 10–70× slower than modes 1–2.

8 One-page summary

Dynamics (L06)

- $h = \frac{1}{\tau}e^{-t/\tau}$, $H(s) = \frac{1}{1+s\tau}$, $\tau\dot{x} = -x + v$.
- $\dot{x} = \frac{1}{\tau}(g(u) + f(x) - x)$.
- $A = (A' - I)/\tau$, $B = B'/\tau$; $A' = \tau A + I$, $B' = \tau B$.
- Identity recurrence with input transform 1: $\dot{x} = u/\tau$.
- Fixed points: $\dot{x} = 0$; stable if $\dot{x}$ goes $+ \to -$.

Decision (PN4 §1)

- $x = 10ut$; reaches 0.9 at $t^* = 0.09/|u|$.
- Saturation: $x - \hat{f}(x) = u$ beyond radius.
- Noise ⇒ drift-diffusion: spread, skew, errors.
- Intercepts in $[0.4, 0.9]$: all silent ⇒ output exactly 0.
- 2-D: radius $\geq \sqrt{2}$, no intercept trick, mixed selectivity.

Functions and LMU (L07)

- Nyquist $N = 2BT$. Orthonormal: $x_i = \langle \phi_i, f \rangle$.
- $\tilde{P}_0 = 1$, $\tilde{P}_1 = 2r-1$, $\tilde{P}_2 = 6r^2-6r+1$; $\tilde{P}_i(0) = (-1)^i$, $\tilde{P}_i(1) = 1$.
- $\theta A_{ij} = (2i+1)\{-1; (-1)^{i-j+1}\}$, $\theta B_i = (2i+1)(-1)^i$.
- $q = 1$: $\dot{x} = (u - x)/\theta$, low-pass, Padé [0/1].
- $u(t - \theta') \approx \sum_i \tilde{P}_i(\theta'/\theta)x_i$.
- RMSE vs q: U-shape. vs neurons: falls, levels.

Supervised learning (L08)

- $\epsilon = \hat{y} - y^{\mathrm{d}}$, $\Delta d_j = -\eta\epsilon a_j$.
- PES: $\Delta w_{ij} = -\eta\alpha_i\langle \boldsymbol{\epsilon}, \mathbf{e}_i \rangle a_j$.
- Reverse sign ⇒ gradient ascent, runaway.
- Floor > 0: neurons, noise, jitter, lag.
- PD offset $-mgL\cos\theta^*/k_p = -0.2$; learns $mgL\cos\theta$; $\epsilon = -\tau_{PD}$.

Unsupervised (L08)

- PCA: eigenvectors of covariance; SVD $\mathbf{V}$.
- Hebb $\eta a_i a_j$ (unstable). Oja $\eta(a_i a_j - a_i^2 w_{ij}) \to$ PC 1.
- BCM (L08): $\eta a_j a_i(\theta - a_i)$.
- STDP: pre→post LTP, post→pre LTD, ∼10 ms.

SVD (L09)

- $\gamma = \mathbf{A}^T\mathbf{A} = \mathbf{USU}^T$ (neurons × neurons), shared by all functions.
- $\chi = \mathbf{AU}$, $\Phi = \mathbf{U}^T\mathbf{D}$, $\chi^T\chi = \mathbf{S}$.
- $\chi_m \approx$ Legendre (heterogeneous LIF).
- $N_S(l, D) = \binom{l+D-1}{l}$.
- $\Phi_m = c_m/(S_m + \sigma^2)$; $\Delta E_m = \frac{c_m^2}{S_m}(\frac{\sigma^2}{S_m} - 1)$.
- Usable iff $S_m > \sigma^2$. PES mode rate ηS_m.

Sketches to rehearse

1. Accumulator ramps for $\pm 0.1, \pm 0.2$, levelling just past ± 1; $\hat{f}(x)$ vs x.
2. Drift-diffusion trace hitting a bound.
3. choice(x) ideal vs decoded; intercept tuning curves.
4. $q = 1$ step response vs true delay; RMSE vs q and vs neurons.
5. PES chunk RMSE for two η; pendulum PD vs adaptive.
6. STDP window; singular spectrum (log) with σ^2 floor; χ_1–χ_4.

How to answer

- Explanations: cause + condition + observable consequence.
- Label sketch axes, thresholds, key values.
- State sign conventions (ϵ, $\mathbf{A}$ orientation).
- About 1.5 minutes per mark; sketches first.